LogicFlow

Full Project Architecture, Pipeline, File Roles, and Connections

Evidence-based review of the workspace at c:/projects/major project final

Generated 2026-09-08

Purpose: document what the project contains, how execution flows through it, how each module connects, what each role does, and which parts are complete, optional, or currently disconnected.

1. Executive Summary

- LogicFlow is an autonomous AI coding-agent prototype. Its intended loop is: read a GitHub issue, locate the affected source file using a FAISS code index, ask a local Ollama model for a complete patch, compile-test the patch in Docker, run a lightweight OWASP-pattern review, and create a GitHub pull request.
- There are two user-facing entry points: the command-line interface in main.py and the Streamlit dashboard in dashboard/streamlit_app.py. Both construct the same LogicFlowAgent from agents/core.py.
- The central orchestration object is LogicFlowAgent. It owns run state and delegates retrieval to FaissIndexer, isolated compilation to DockerSandbox, and issue/PR work to GitHubManager.
- The repository includes a small calculator demo with a known failing behavior: multiply() returns a + b. The test suite cannot currently collect in the active interpreter because openai is not installed.
- Several components are support code rather than active pipeline stages: tools/ast_tool.py is not imported by the agent, tools/init.py is not a package initializer, and cofig.py duplicates config.py under a misspelled filename.

2. System at a Glance

Primary runtime flow

```
User -> main.py or Streamlit -> LogicFlowAgent -> GitHubManager -> FaissIndexer ->
Ollama/OpenAI-compatible API -> DockerSandbox -> security checks -> GitHubManager -> PR or fallback git
commands
```

Indexing flow

```
Workspace files -> language detection -> boundary-aware chunks -> SentenceTransformer embeddings ->
FAISS IndexFlatL2 -> code_index.faiss + chunks_metadata.pkl
```

Layer	Implemented by	Responsibility
Entry points	main.py; dashboard/streamlit_app.py	CLI commands and visual controls.
Configuration	config.py	Dataclass settings, environment overrides, language/compiler maps.
Orchestration	agents/core.py	State machine, retries, LLM prompt, validation, security, outcome.
Retrieval	tools/faiss_tool.py	Index, semantic search, full target content, trimmed dependencies.
Validation	tools/docker_test_tool.py	Language-specific Docker compilation with limits.
GitHub	tools/github_tool.py	Issue parsing, branch, file commit, PR, fallback commands.
Optional helper	tools/ast_tool.py	Python AST extraction/replacement; currently not wired in.

3. End-to-End Pipeline

Stage	What happens
1. Configure	get_config() creates one global LogicFlowConfig from defaults and selected environment variables. The dashboard can mutate the same object through controls.
2. Prepare the index	initialize_index() first attempts load_index(). If unavailable, build_index() scans the configured workspace, skips common generated/hidden folders, chunks supported source files, embeds chunks, and persists FAISS plus metadata.
3. Acquire the issue	process_issue() asks GitHubManager.fetch_issue_by_number(). GitHubManager parses body text for file paths, errors, code blocks, labels, and metadata.
4. Select the target	An explicit override wins. Otherwise the agent matches referenced issue files against indexed paths, then searches FAISS using error text, then issue title.

Stage	What happens
5. Build context	FaissIndexer.get_context_for_fix() reads the complete target file and retrieves up to eight other relevant files. ContextTrimmer keeps structural signatures, types, imports, macros, and declarations to reduce prompt size.
6. Generate a patch	The OpenAI client is configured against Ollama's OpenAI-compatible endpoint. The model must return JSON containing patches, explanation, security_audit, and is_secure. The parser handles plain JSON, code fences, regex extraction, single quotes, Python literals, and a code-block fallback.
7. Retry after compile failure	The agent permits two attempts. A failed Docker result becomes feedback in the retry system prompt. A successful compile exits the loop.
8. Review security	The model's audit is combined with regex checks for eval, exec, system, popen, SQL concatenation, hardcoded secrets, pickle, unsafe YAML, subprocess patterns, os.system, innerHTML, and document.write.
9. Publish	GitHubManager creates a timestamped branch, updates or creates the target file, then opens a PR and labels it. If the API is unavailable or rate-limited, it returns manual git/gh commands.
10. Display outcome	CLI prints AgentResult and logs. Dashboard stores the result in Streamlit session state and shows summary, security status, explanation, audit, fallback commands, and patched code.

4. File-by-File Responsibilities

Path	Role
main.py	CLI entry point. Defines index, fix, status, and dashboard commands; initializes the agent; prints results; launches Streamlit as a subprocess.
config.py	Canonical configuration. Holds Ollama, workspace, FAISS, Docker, and GitHub dataclasses plus language extension, image, and compile-command maps.
cofig.py	Byte-for-byte duplicate of config.py under a typo. It is not imported by the visible application and creates maintenance ambiguity.
agents/__init__.py	Package marker; no visible runtime behavior.
agents/core.py	Core orchestration. AgentState tracks issue, target, code, attempts, sandbox result, PR result, logs, and status. AgentResult is the public outcome contract.
tools/faiss_tool.py	RAG/index layer. CodeChunk stores chunk metadata. FaissIndexer builds/loads/searches the vector index and supplies target/dependency context. ContextTrimmer extracts structural lines.
tools/docker_test_tool.py	Sandbox layer. Writes candidate code into a temporary directory and invokes docker run with memory, CPU, optional network isolation, language image, and compiler command.
tools/github_tool.py	Integration layer. GitHubIssue and PullRequestResult are internal data contracts. GitHubManager talks to PyGithub and degrades to generated commands.
tools/ast_tool.py	Standalone Python AST utility. Finds functions/classes, extracts traceback targets, and replaces functions. No import from core.py currently connects it to production flow.

Path	Role
tools/init.py	Named init.py rather than __init__.py. It does not make tools a conventional Python package; imports currently work through namespace-package behavior in modern Python.
dashboard/streamlit_app.py	Interactive operator UI. Handles session state, settings, status checks, indexing, issue selection, execution, logs, and results.
test_llm.py	Manual Ollama smoke script, not a deterministic unit test. It calls the local model directly and prints raw output.
requirements.txt	Declares OpenAI client, HTTP, FAISS, embeddings, GitHub, Streamlit, and NumPy dependencies.
workspace/logicflow-test-repo/calculator.py	Demo target repository source. Contains add, subtract, multiply, and divide; multiply is intentionally/accidentally incorrect.
workspace/logicflow-test-repo/test_calculator.py	Pytest checks for the calculator. The multiply assertion currently fails once collection dependencies are satisfied.
faiss_index/code_index.faiss	Persisted FAISS vector index artifact. The matching chunks_metadata.pkl file expected by load_index() is not visible in the workspace listing.
README.md	Currently empty; no setup, architecture, usage, or operational documentation is supplied by the project itself.
logicflow/	Directory present in the workspace listing, but no source files were visible in the supplied project tree; its purpose cannot be verified from the current files.

5. Connection Map

The important imports and call relationships are:

- main.py imports config, agents.core, and tools.faiss_tool; its fix command calls LogicFlowAgent.initialize_index() and process_issue().
- dashboard/streamlit_app.py imports config, FaissIndexer, DockerSandbox, GitHubManager, and LogicFlowAgent; UI buttons call those same services directly or through the agent.
- agents/core.py imports config plus FaissIndexer, DockerSandbox, GitHubManager, GitHubIssue, PullRequestResult, and SandboxResult. This is the controlling dependency hub.
- FaissIndexer depends on SentenceTransformer, FAISS, NumPy, and config. It writes code_index.faiss and chunks_metadata.pkl beneath the configured index path.
- DockerSandbox depends on subprocess and config. It invokes Docker images and compiler commands selected by file extension.
- GitHubManager lazily imports PyGithub, uses GITHUB_TOKEN/GITHUB_REPO, and returns domain dataclasses instead of exposing PyGithub objects to the agent.
- The local model connection uses the OpenAI SDK but points base_url to Ollama. No remote OpenAI service is required if Ollama is running locally with the configured model.
- ASTExtractor currently has no caller in the production graph. It could become a targeted Python patching stage, but today patches replace the complete target file content.

Connection diagram in plain form

```
CLI / Dashboard -> Config -> Agent -> {GitHub issue, FAISS context, Ollama patch, Docker compile, OWASP checks, GitHub PR} -> Result/UI
```

6. Roles

Role	Owner	What the role does
Operator	CLI user or dashboard user	Sets configuration, prepares an index, selects an issue, and launches processing.
Orchestrator	LogicFlowAgent	Controls the lifecycle, retries, state transitions, and final result.
Retriever	FaissIndexer / ContextTrimmer	Finds relevant code and supplies compact dependency context.
Patch generator	Ollama model through OpenAI SDK	Produces complete replacement file content and a security explanation.
Compiler gate	DockerSandbox	Checks syntax/compilation in a bounded container.
Security gate	LogicFlowAgent._evaluate_security	Combines model audit with simple dangerous-pattern detection.
Repository integrator	GitHubManager	Fetches issues and publishes fixes as branches, commits, and PRs.
Fallback operator	Generated git commands	Provides a manual route when GitHub API work cannot complete.
Demo/test role	calculator.py and test_calculator.py	Provides a small reproducible bug target and expected behavior.

7. Configuration and Runtime Requirements

Area	Defaults / inputs	Required for
Ollama	http://localhost:11434/v1; llama3.2:3b; 300s request timeout	LLM patch generation and test_llm.py.
Workspace	./workspace; supported source extensions; 500 KB max file	Index building and target file reads.
FAISS	./faiss_index; all-MiniLM-L6-v2; 512-line chunks; overlap 50	Loading or building semantic retrieval.
Docker	120s; 512m; 50% CPU quota; network disabled	Compilation gate.
GitHub	GITHUB_TOKEN, GITHUB_REPO, optional GITHUB_BASE_BRANCH	Issue fetching and PR publication.

- Environment variables read by config.py: OLLAMA_BASE_URL, OLLAMA_MODEL, OLLAMA_TIMEOUT, OLLAMA_CONNECT_TIMEOUT, LOGICFLOW_WORKSPACE, LOGICFLOW_FAISS_PATH, LOGICFLOW_EMBEDDING_MODEL, GITHUB_TOKEN, GITHUB_REPO, and GITHUB_BASE_BRANCH.
- The FAISS loader expects both code_index.faiss and chunks_metadata.pkl. A lone code_index.faiss is insufficient for a successful load.
- Docker must be installed and running, and the selected language image must be pullable. Network isolation is applied to containers by default.
- The Python dependencies in requirements.txt are not installed in the active interpreter used for the baseline test; specifically, importing openai failed.

8. Current Findings and Risks

Priority	Finding	Evidence and impact
High	Test collection blocked	test_llm.py imports openai at collection time, but openai is absent from the active interpreter. Install requirements in the intended virtual environment or isolate the manual smoke script from pytest discovery.
High	Demo behavior fails	calculator.multiply(3, 4) returns 7, while test_calculator.py expects 12. This is the clearest reproducible bug target.
Medium	Index artifact incomplete	load_index() requires chunks_metadata.pkl in addition to code_index.faiss; only the FAISS file was visible.
Medium	AST helper disconnected	ASTExtractor can narrow Python edits but is never called by LogicFlowAgent, so the active path asks for complete-file replacement.
Medium	Security review is heuristic	Regex checks can miss context-sensitive vulnerabilities and can also flag benign text. It should be treated as an advisory gate, not a complete security analysis.
Medium	PR result semantics	process_issue() returns success=True after a working patch even when PR creation fails, provided fallback commands exist or the patch passed compilation.
Low	Duplicate configuration	cofig.py and config.py can drift and confuse maintainers; the correctly named config.py is the active import target.
Low	Documentation gap	README.md is empty, so setup, required services, commands, and limitations are undocumented outside source comments.

9. Simplest Working Demonstration

- The smallest local logic check is the calculator test: it exercises no Ollama, FAISS, Docker, or GitHub and should be made green first.
- The smallest agent path is process_issue_from_raw(), which avoids GitHub issue fetching and FAISS target selection but still requires Ollama and Docker because it generates and compiles a patch.
- The full path is main.py fix , after an index is available and both Ollama and GitHub are configured.
- The dashboard path is main.py dashboard, which launches Streamlit and exposes status checks, index controls, issue selection, and execution results.

10. Recommended Operating Sequence

1. Install dependencies into the project virtual environment and verify imports.
2. Start Ollama and pull the configured model; verify the model responds to the OpenAI-compatible endpoint.
3. Start Docker and verify the selected compiler image can run.
4. Fix or intentionally use the calculator demo issue as a local test target.

```
5. Build the index with python main.py index --workspace ./workspace --output ./faiss_index.
```

6. Configure GITHUB_TOKEN and GITHUB_REPO only when testing real issue/PR automation.

```
7. Run python main.py status, then use python main.py dashboard or python main.py fix .
```

8. Review the generated patch, sandbox result, security audit, and PR/fallback output before merging.

11. Final Assessment

LogicFlow has a coherent prototype architecture centered on a useful self-healing loop. The strongest design choice is the separation between orchestration and specialized services: retrieval, sandboxing, and GitHub integration each have a clear owner. The main maturity gaps are operational rather than conceptual: dependency setup is not verified,

the persisted index artifact appears incomplete, the AST utility is unused, security analysis is heuristic, and project documentation is absent. The calculator repository provides a good minimal fixture for validating the intended repair workflow once the environment is prepared.

This report was generated from the source files and workspace artifacts present at report-generation time. It does not claim runtime success for external services that were not available during inspection.